C++ STL — `std::stack` Cheat Sheet

Purpose: A self-contained DSA / Competitive Programming / Interview reference. The goal is to make stack revision possible from this document alone.

1. Core Idea

Stack = LIFO (Last In, First Out)

The last element inserted is the first element removed. Think of a stack of plates: you add and remove plates from the top.

Mental model:

```
top → [ 30 ]
[ 20 ]
[ 10 ]
```

If 40 is pushed, it becomes the new top. If pop() is called, 40 is removed first.

Pattern: When the *most recent unresolved/opened item* must be handled first → think **stack**.

2. Why std::stack Exists

You could use a vector and manually restrict yourself to one end. But `std::stack` packages the desired behavior into a clean interface: **LIFO only**. This communicates intent and avoids exposing general-purpose operations that a stack does not need.

Conceptually:

```
std::stack
↓
LIFO interface / behavior
↓
underlying container stores the elements
```

Important: `std::stack` is a **container adapter**, not a general-purpose container like vector.

3. Declaration

```
stack st;
```

stack<int> = stack type storing integers.
st = variable name.

Typical header:

```
#include <stack>
using namespace std;
```

4. Core Operations

Operation	What it does	Complexity
st.push(x)	Adds x to the top.	O(1)
st.top()	Accesses the top element; does not remove it.	O(1)
st.pop()	Removes the top element; returns nothing.	O(1)
st.empty()	Checks whether the stack has no elements.	O(1)
st.size()	Returns the number of elements.	O(1)

5. The Most Important Distinction: top() vs pop()

`top()` **looks at** the top element. `pop()` **removes** the top element.

```
st.top(); // access
st.pop(); // remove
```

Why doesn't pop() return the removed value? The interface separates observation from modification. If you need the value, read it first, then remove it:

```
int x = st.top();
st.pop();
```

This is the standard pattern when you need both the value and removal.

6. Example: Predict the Stack

```
stack st;
st.push(10);
st.push(20);
st.push(30);

cout << st.top();
st.pop();
cout << st.top();
```

Output: 30 20

Before pop: top → 30 → 20 → 10. Removing 30 exposes 20 as the new top.

7. Safety Rule

Never call top() or pop() on an empty stack. Check first when emptiness is possible:

```
if (!st.empty()) {
cout << st.top();
st.pop();
}
```

Calling `top()` or `pop()` when the stack is empty results in **undefined behavior**.

8. What std::stack Does NOT Give You

• No random access: you cannot do st[3].

• No begin()/end() iteration interface.

• You work through the top of the stack.

• The abstraction intentionally exposes only operations needed for stack behavior.

9. Underlying Container — Just Enough to Know

`std::stack` is an adapter over an underlying container. The default underlying container is typically **std::deque**. A suitable container such as **std::vector** can also be used when its required operations are available.

Example:

```
stack st; // default underlying container
stack> st; // vector-backed stack
```

For current DSA / CP goals, remember the abstraction first: **stack = LIFO behavior**. You do not need implementation-level details to use it effectively.

10. Complexity

The core operations `push()`, `pop()`, `top()`, `empty()`, and `size()` are generally **O(1)**. The stack operates at its accessible end rather than searching through elements.

Space: O(n) for n stored elements.

11. DSA / CP Pattern Recognition

Think stack when:

Problem clue	Think
Most recent item must be processed first	LIFO → stack
Most recently opened/unresolved item must be matched	Balanced parentheses / brackets
Need to undo the most recent operation	Undo-style behavior
Need to simulate nested processing	Stack-like state

Balanced brackets: when you encounter an opening bracket, remember it. When a closing bracket appears, it must match the most recently opened unmatched bracket. That is exactly LIFO behavior.

12. Stack vs Vector

vector	stack
General-purpose sequence container	Container adapter
Random access available	No random access
Flexible sequence operations	Restricted LIFO interface
Indexes / iterators available	Operate through top
Use for general sequence needs	Use when you specifically need LIFO behavior

13. Common Mistakes

• **Confusing top() and pop():** top reads; pop removes.

• **Expecting pop() to return the removed element:** save top() first if needed.

• **Calling top()/pop() on an empty stack:** check empty() first.

• **Thinking stack has indexes:** stack deliberately does not expose random access.

• **Confusing LIFO with FIFO:** stack = newest first; queue = oldest first.

• **Overthinking implementation:** for current goals, focus on the LIFO abstraction and operations.

14. Interview Questions — Quick Answers

Q: What is a stack?
A: A data structure/abstraction following LIFO: Last In, First Out.

Q: Why use std::stack instead of vector?
A: To express stack-style LIFO access through a simple interface rather than exposing general sequence operations.

Q: What does top() do?
A: Returns a reference to the current top element without removing it.

Q: What does pop() do?
A: Removes the current top element and returns void.

Q: Why are top() and pop() separate?
A: You can inspect without removing; if you need both the value and removal, read top() first, then pop().

Q: What happens if top()/pop() is called on an empty stack?
A: Undefined behavior.

Q: What is a container adapter?
A: A class that provides a restricted interface/behavior on top of an underlying container.

Q: Typical complexity?
A: Core operations are generally O(1); n stored elements require O(n) space.

15. Mini Code Template

```
#include
using namespace std;

int main() {
stack st;

st.push(10);
st.push(20);
st.push(30);

if (!st.empty()) {
cout << st.top();
st.pop();
}
}
```

16. One-Page Memory Map

```
STACK
↓
LIFO — Last In, First Out
↓
```

```
Most recent item comes out first

push(x) → add to top
top() → look at top
pop() → remove top
empty() → check whether empty
size() → number of elements

Core operations → generally O(1)
Space → O(n)
No indexing / random access
Container adapter

Problem clue: most recent unresolved item first → STACK
```

17. Final Takeaway

Do not memorize stack as a collection of functions. Remember the behavior first:

Need LIFO behavior → use stack.
Need to inspect the newest element → top().
Need to remove the newest element → pop().
Need to add a new element → push().
Need safety before accessing/removing → empty().

Advanced topics such as monotonic stacks can be learned later when they appear naturally in DSA.